

"长沙银行杯"2026中国研究生创芯大赛·EDA精英挑战赛赛题指南

赛题：基于 AI 的覆盖率驱动通用测试激励自动生成

目录

- 第一部分 命题文档
- 第二部分 附件 A：覆盖组定义规范
- 第三部分 附件 B：Docker 提交规范与推理接口标准
- 第四部分 附件 C：覆盖率模拟工具规范

第一部分 命题文档

一、赛题名称

基于AI的覆盖率驱动通用测试激励自动生成

二、命题单位

湖南泛联新安信息科技有限公司

三、赛题背景

功能验证是芯片设计流程中耗时最长的环节之一，其中测试激励的生成质量直接影响验证效率和覆盖率达到收敛的速度。当前业界主流方法依赖工程师手动编写定向测试或使用受约束随机测试（CRT），存在以下痛点：

- 手动编写激励效率低，难以覆盖边界情况；
- 随机测试存在大量冗余激励，在复杂设计上覆盖率收敛缓慢；
- 缺乏自适应的反馈机制，无法根据已覆盖情况动态调整激励策略；
- 针对特定设计优化的激励生成方法难以迁移到不同类型的被测设计。

近年来，强化学习、进化算法、大语言模型等AI技术在芯片设计领域展现出强大能力，为激励自动生成提供了新的思路：将DUT视为环境，激励作为动作序列，覆盖率作为奖励信号，通过与DUT交互自主学习高效的激励策略，加速覆盖率收敛。

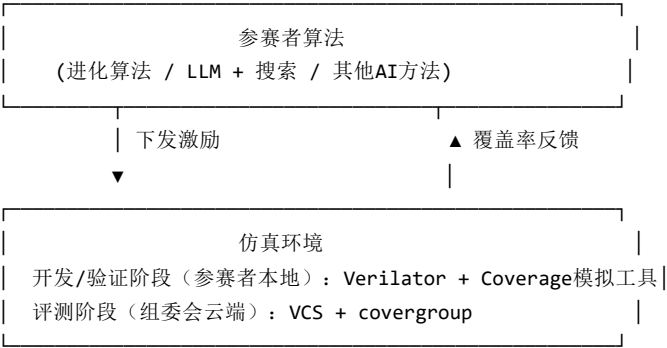
本题要求参赛者设计并实现一种**自动激励生成方法**，在组委会提供的多种不同类型被测设计（DUT）上，以覆盖率为优化目标，在给定的仿真预算内达到尽可能高的覆盖率。

四、赛题描述

4.1 目标

设计并实现一套自动测试激励生成系统，以覆盖率指标为核心反馈信号，在多个不同类型的DUT上自动生成高质量仿真激励序列。方法应具备跨DUT的泛化能力，在覆盖率收敛效率和最终覆盖率达到显著优于随机基线方法。

4.2 赛题框架



关键说明：

- 参赛者在**开发/验证阶段**可使用任意开源工具在本地开发和验证方法
- **评测阶段**，参赛者提交的推理镜像将在组委会云端环境运行，使用商用仿真器（VCS）采集 SystemVerilog 功能覆盖率（covergroup/cross）
- 评测阶段可以接收覆盖率反馈信息用于推理决策，但**严格禁止修改模型权重参数**；允许在推理时根据反馈动态调整prompt策略或 Agent规划（即prompt engineering层面的适应性调整）
- 方法不限类别，鼓励结合大语言模型等AI技术

4.3 组委会提供

4.3.1 DUT 集合

公开 DUT（赛前公开，用于方法开发与本地验证，DUT以实际发布为准）：

- **3 个**不同类型的数字逻辑 DUT 设计文件（Verilog），按难度分为三档：

难度档	DUT 名称	特征	bin 数
低	AES 单轮加密运算	多阶段 FSM + 结果正确性断言（result_ok）+ 数据边界	35
最高	SPI 主设备协议控制器	覆盖点绝大多数依赖"完成完整传输序列"；各模式窗口长度隐藏	94
高	温控状态机控制器	时序窗口 + 数据依赖（温差温区）+ 多复位源 + 正确输出断言（drv_ok）	26

- 每个 DUT 附带功能规格说明文档（DUT spec）、covergroup 定义文件、CoverageSimulator 配置
- 公开 DUT 的 RTL 可直接获取（用于训练/验证方法）；关键隐蔽参数（如各 mode 窗口长度、温区范围、正确性判据）为隐藏参数，具体值不随包公布，参赛者须以覆盖率反馈探测
- DUT 规格范围：RTL 代码量约 100~3000 行，状态寄存器数 20~200 个
- 所有 DUT 使用 BSD / MIT / Apache 等开放许可，文档注明来源和许可证

评测 DUT（赛前保密）：

- 3 个用于正式评分的 DUT，赛前不公布
- 评测时，各 DUT 的功能规格说明文档与覆盖组定义文件通过标准接口输入给推理镜像

4.3.2 覆盖率定义与采集

核心原则：开发和评测阶段使用**相同的覆盖点定义**，确保策略学习的有效性和一致性。覆盖组（covergroup）是一组覆盖点（coverpoint）和交叉覆盖点（cross）的集合，覆盖点再细分为一个个 bin（如操作码分类值）。每个覆盖点的 bin 命中状态构成覆盖率反馈向量。覆盖点、覆盖组、bin 等术语的完整定义及格式规范详见附件 A。

开发/验证阶段（参赛者本地）：

- 组委会提供**覆盖率模拟工具**（Coverage Simulator），一个 Python 工具包
- 该工具读取覆盖点定义，在 Verilator 仿真过程中观测 DUT 信号，按覆盖组规则计算各覆盖点的命中状态，输出与评测阶段一致的 numpy 向量格式
- 工具实现详见附件 C

评测阶段（组委会云端）：

- 使用商用仿真器（VCS）的 SystemVerilog covergroup 原生采集功能覆盖率（含基本覆盖点、交叉覆盖点等）
- 每个 DUT 配套 covergroup.svh 定义文件，覆盖点总 bin 数：26~120 个（含交叉覆盖点），按难度分层
- 每步仿真后返回标准覆盖率反馈（格式见附件A）

为什么分两阶段：

- 开发阶段用 Verilator + 模拟工具：免费、快速、可本地运行
- 评测阶段用 VCS：覆盖组采集结果权威、与工业界一致
- 两阶段覆盖定义相同，仅采集实现不同——组委会在赛前完成双工具交叉验证
- Docker 镜像提交规范和推理接口标准详见附件 B

4.3.3 仿真环境与算力

阶段	仿真工具	算力	由谁提供
开发/验证	Verilator + Coverage模拟工具	参赛者自行解决	参赛者
评测	商用仿真器（VCS）	组委会云端 CPU 集群	组委会

评测墙钟时间规定（以大赛组委会正式公告为准）：

- 全流程（含 Docker 启动、DUT 挂载、3 个评测 DUT 完整运行）不超过 **2 小时**
- 时间预算分配参考：Docker 启动 ~2min + DUT 挂载 ~1min + 3 个 DUT × 各约 37min；对应每 cycle 约 44ms（含 VCS 仿真 + predict + 数据交互）
- 超时则取当前已完成的 DUT 覆盖率结果作为最终成绩
- 墙钟时间不作为评分因子，仅用于保障评测流程的效率

评测环境网络：

- 允许外网访问（供 LLM API 调用）
- 不保证 GPU
- 使用云端 LLM API 的参赛者需在报告中说明预期单次 API 调用耗时，评测时 API 耗时计入墙钟时间上限
- 评测阶段每个 DUT 的 LLM API 调用总 Token（输入+输出）上限 **50 万 Token**，由仿真适配层统计；超限后拒绝调用，参赛者需实现降级策略

4.3.4 基线参考与 Docker 基础镜像

- 组委会提供**Docker 基础镜像**，包含 InferenceInterface 骨架代码、覆盖率模拟工具，参赛队在此基础镜像之上开发自己的算法

- 基础镜像内置纯随机激励生成和简单贪心搜索两种基线方法（供参赛者本地参考）
- 随机基线等组委会评分参照数据属评测侧材料，不随发布包公布

4.4 仿真预算

阶段	单个DUT仿真预算参考	说明
开发/ 验证阶段	不限（参赛者本地）	鼓励在合理范围内充分开发
评测阶段	5万~10万 cycles	具体额度以大赛组委会正式公告为准；若有 DUT 在 2 小时墙钟内未跑完指定 cycles，以墙钟超时时的当前覆盖率作为最终结果（以先到达的条件为准）

4.5 输入

项目	说明
公开 DUT RTL	3 个多种类型数字逻辑设计（含RTL + spec + covergroup定义），赛前公开
DUT 功能规格说明	每个 DUT 的功能描述、接口说明、操作模式
覆盖组定义	每个 DUT 的 covergroup.svh 和 coverage_meta.json
覆盖率模拟工具	组委会提供的 Python 工具包，在 Verilator 端输出与评测一致的覆盖率反馈
Docker 基础镜像	组委会提供的开发起点（含推理接口骨架 + 基线方法）

4.6 输出

参赛队在组委会提供的**Docker 基础镜像**之上构建并提交 **Docker 镜像**（不暴露源码），通过以下标准接口与评测环境交互：

```
class InferenceInterface:
    def __init__(self, dut_spec_path: str, covergroup_path: str):
        """接收DUT规格说明和覆盖组定义文件路径"""

    def predict(self, coverage_state: np.ndarray,
               step: int, max_steps: int) -> np.ndarray:
        """根据当前覆盖率状态返回下一个激励动作

    Args:
        coverage_state: 当前各覆盖点已覆盖状态（二值向量，shape=(N,)，dtype=float32）
        step:          当前仿真步数（0-indexed）
        max_steps:     最大仿真步数

    Returns:
        action: 下一个激励动作，一维 numpy 向量（shape=(D,)，dtype=float32）
               D 为动作空间维度，由 DUT spec 定义
               评测脚本中的仿真适配层负责将 action 解码为 DUT 引脚激励

    """
```

评测约束：

- predict() 内部**不得修改模型权重参数**；允许根据覆盖率反馈动态调整prompt策略或Agent规划
- 可以读取 coverage_state、step 等输入信息进行推理决策
- action 向量的维度 D 由 DUT spec 定义，可理解为动作向量中独立控制通道的数量。对于多参数激励（如操作码 + 多个操作数），所有参数展开为一维向量，解码规则见 DUT spec

覆盖率反馈信息的使用：

coverage_state 为二值向量（1.0 表示已覆盖），参赛者可从中提取以下反馈信息：

```
# 当前总覆盖率
total_cov = float(np.sum(coverage_state)) / len(coverage_state)

# 上一步新增的 bin 数（需保存上一轮 state）
delta_bins = int(np.sum(coverage_state)) - prev_covered
reward = delta_bins / total_bins # 可作为 RL 奖励信号

# 分类型覆盖率（需配合 coverage_meta.json）
# coverage_state 的 bin 顺序按 coverage_meta.json 中 coverpoints 数组依次拼接
```

RL 算法可将覆盖率增量作为奖励信号、二值向量作为状态输入。通过保存历史 state，可获知完整的时间差分信息。

除 Docker 镜像外，需提交：

- 1. **覆盖率报告**：在公开 DUT 上的覆盖率收敛曲线，含3次独立重复实验的均值±方差
- 2. **设计文档**：方法说明，包括算法架构、状态/动作空间设计、方法策略
- 3. **消融实验报告**：至少 3 种配置的对比结果（如学习率、网络深度/宽度、奖励函数权重、探索策略参数、LLM prompt 策略等——围绕参赛者所采用方法的关键设计变量展开）
- 4. **LLM使用报告**（如使用）：模型名称、版本、调用方式（本地/API）、在方法中的角色
- 5. **README**：Docker镜像使用说明

4.7 LLM 使用规范

要求	说明
指定模型	DeepSeek-V4 （开源本地部署或云端 API 均可）；也接受 GLM-4 系列、豆包（Doubao）等国产模型作为替代，但评测环境默认提供 DeepSeek-V4 API 端点和 Token 配额
使用报告	必须报告：模型名称、版本、调用方式（本地/API）、在方法中的角色
Token 限制	评测阶段每个 DUT 的 LLM API 调用总 Token（含输入和输出）上限为 50 万 Token ，由评测脚本在仿真适配层中统计；超出部分拒绝调用并计入日志，predict() 需自行处理 API 调用失败的降级策略
公平性	LLM不作为独立评分项，最终排名仅由覆盖率公式决定

4.8 开发建议

- 开发/验证算力由参赛者自行解决，组委会不提供开发环境算力
- 参考耗时（基于 Intel i7-12700, 8 核, Verilator 5.x, 随机方法）：单个 DUT 每 1 万 cycles 仿真约 30~60 秒
- 参赛者应自行评估方法和算力的可行性

Verilator 安装参考：

- Linux (Ubuntu/Debian): sudo apt install verilator （系统包管理器≥5.0 即可）；如需最新版可从源码编译：
git clone https://github.com/verilator/verilator && cd verilator && autoconf && ./configure && make -j && sudo make install
- Windows: 推荐使用 WSL2（Windows Subsystem for Linux）安装 Ubuntu，然后按 Linux 方式安装
- macOS: brew install verilator
- Verilator 的 Python 绑定可参考 verilator 项目的 --cc + --prefix 选项生成共享库，或通过 pyverilator 等第三方工具与 Python 交互

动作空间缩减策略建议：

- 实际 DUT 的激励空间通常远大于示例，直接使用全空间离散 RL 会出现稀疏奖励问题
- 建议采用以下一种或多种策略缩减动作空间：
 - **分层生成**：先选择激励类型（事务类型），再在受限范围内选择参数
 - **语义约束**：基于 DUT spec 过滤非法组合，缩减有效动作范围
 - **LLM 辅助**：利用 LLM 解析 spec 中的语义信息，生成合理操作数或事务参数
 - **连续参数化**：对操作数维度使用连续动作空间 + 离散化

五、评分标准（总分100分）

5.1 客观评分公式

评分基于评测 DUT 上达到的功能覆盖率，采用**基线减法归一化**以拉开区分度，计算公式如下：

$$Score = \frac{1}{N} \sum_{i=1}^N \left(0.6 \times \frac{C_{final}^{(i)} - C_{baseline}^{(i)}}{C_{max}^{(i)} - C_{baseline}^{(i)}} + 0.4 \times \frac{AUC^{(i)} - AUC_{baseline}^{(i)}}{AUC_{max}^{(i)} - AUC_{baseline}^{(i)}} \right) \times 100$$

- $N = 3$ ：评测 DUT 数量
- $C_{final}^{(i)}$ ：参赛者在 DUT i 上的覆盖率终值
- $C_{baseline}^{(i)}$ ：随机基线方法在 DUT i 上的覆盖率终值（由组委会预跑确定并公布；与参赛者一致的仿真预算条件下，固定随机种子，每 1000 cycles 采点记录覆盖率曲线，基线数据在赛题发布时随基础镜像一同公布）
- $C_{max}^{(i)}$ ：当前 DUT i 上所有参赛者的覆盖率终值最大值
- $AUC^{(i)}$ ：参赛者的覆盖率-仿真预算曲线下面积（梯形法积分，衡量收敛速度）
- $AUC_{baseline}^{(i)}$ ：随机基线方法的 AUC（由组委会基于与参赛者一致的 1000 cycles 采点间隔计算并公布，基线数据随基础镜像一同发布）
- $AUC_{max}^{(i)}$ ：当前 DUT i 上所有参赛者的 AUC 最大值
- 分子项若为负值则取 0（不得分）

基线减法归一化的意义：传统组内最大值归一化仅消除 DUT 间难度差异，当多支队伍在简单 DUT 上均接近满分时难以区分。引入基线减法后，只有显著超越随机基线的队伍才能获得高分，拉开区分度。

保障措施：

- 若某个 DUT 上所有参赛者的 $C_{max} \leq C_{baseline} + 0.02$ （即无人显著超出随机基线，0.02 表示绝对覆盖率 2%，非百分比），则以 $C_{baseline} \times 1.5$ 替代 C_{max} 作为覆盖率终值项的归一化分母，即 $C_{max}^{(i)} = \max(C_{max}^{(i)}, C_{baseline}^{(i)} \times 1.5)$
- AUC 项同样设置分母下限：当 $AUC_{max} \leq AUC_{baseline} \times (1 + \alpha)$ 时（其中 $\alpha = 0.02$ ，与覆盖率终值项的 2% 阈值一致），以 $AUC_{baseline} \times 1.5$ 作为 AUC 项的分母下限，即 $AUC_{max}^{(i)} = \max(AUC_{max}^{(i)}, AUC_{baseline}^{(i)} \times 1.5)$
- 两项的保障措施**独立触发**、独立判断，互不影响
- 所有计算**先替换分母值，再计算分式**，分子若为负仍取 0
- 此机制避免 DUT 难度过高导致该 DUT 权重虚高或分数爆炸

其中 AUC 采用梯形法积分计算：

$$AUC = \sum_{k=1}^{K-1} \frac{(C_k + C_{k+1})}{2} \times (t_{k+1} - t_k)$$

- C_k ：第 k 个采样点的覆盖率值
- t_k ：第 k 个采样点的仿真步数（cycle）
- 覆盖率曲线每 1000 cycles 采样一个点

- 若某个 DUT 因墙钟超时而导致不同参赛者的有效 cycles 数不同，AUC 统一计算至所有参赛者中最短有效采样点对应的 cycle 数，以保证比较基础的公平性

区分度分析 ($N = 3$, 3 个评测 DUT):

- DUT 间的难度差异通过基线减法归一化 (减去随机基线后除以 $C_{max} - C_{baseline}$ 和 $AUC_{max} - AUC_{baseline}$) 消除, 每个 DUT 独立贡献
- AUC 维度捕获收敛速度差异——即便多个队伍覆盖率终值接近, 收敛速度仍有区分
- 交叉覆盖点 (难度最高 <10% 随机可达率) 提供额外区分空间

5.2 补充说明

- LLM 使用情况在方法说明中体现, 但不单独计分
- 所有开源 DUT 及软件工具均使用 BSD、MIT、Apache 等开放许可证, 详见各组件源码中的 LICENSE 文件

六、自动化评测流程

1. 参赛者基于组委会提供的 Docker 基础镜像构建并提交 Docker 镜像
2. 组委会后台启动 Docker 容器, 挂载评测 DUT 的功能规格说明 (spec) 与覆盖组定义文件
3. 评测脚本通过标准推理接口循环调用 `predict()`, 驱动 VCS 仿真
4. 记录覆盖率收敛曲线, 自动按公式计算得分
5. 全流程墙钟时间不超过 2 小时

七、参赛要求

- 参赛队伍为1-3名在校研究生, 每队可设1-2名指导教师
- 每位参赛者只能加入一个参赛队、参与一道赛题
- 严禁抄袭、盗用他人成果, 借鉴他人工作须注明来源
- 作品中禁止出现学校、指导教师相关信息

八、软件依赖

开发/验证阶段 (参赛者本地, 推荐):

- Verilator ≥ 5.0
- Python ≥ 3.8
- PyTorch / TensorFlow (自选)
- 国产大语言模型: DeepSeek-V4 (指定); 也可使用 GLM-4 系列 / 豆包 (Doubao)

评测阶段 (组委会云端):

- 商用仿真器 (VCS)
- Docker 运行环境

九、参考文献

- [1] Fast and Effective Coverage-Directed Test Generation on RISC-V Using Reinforcement Learning. IEEE/ACM DAC, 2024.
- [2] Coverage-Directed Test Generation via Reinforcement Learning. IEEE/ACM DATE, 2023.
- [3] RL-Cov: Coverage-Directed Test Generation for Processor Verification via Deep Reinforcement Learning. ASP-DAC, 2024.
- [4] Tasiran, S. and Keutzer, K. Coverage Metrics for Functional Validation of Hardware Designs. IEEE Design & Test of Computers, 2001.
- [5] Fine, S. and Ziv, A. Coverage Directed Test Generation for Functional Verification using Bayesian Networks. DAC, 2003.
- [6] Benjamin, M., et al. A Study in Coverage-Driven Test Generation. DAC, 1999.
- [7] RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, Version 20191213.
- [8] Verilator - Fast Verilog/SystemVerilog Simulator. <https://www.veripool.org/verilator>.
- [9] Samarah, A., et al. An Automated Coverage Directed Test Generation Using Genetic Algorithms. IEEE/ACM DATE, 2006.
- [10] Nativ, G., et al. Cost-Effective Coverage-Driven Verification Using Functional Constraints. IEEE/ACM DATE, 2009.
- [11] Gordon, M., et al. Coverage Closure Acceleration via Machine Learning. DVCon, 2020.
- [12] DeepSeek-V4 Technical Report. <https://github.com/deepseek-ai/DeepSeek-V4>.
- [13] GLM-4 Technical Report. <https://github.com/THUDM/GLM-4>.
- [14] 豆包大模型. <https://www.volcengine.com/product/doubao>.

第二部分 附件 A：覆盖组定义规范

A.1 概述

每个 DUT 配套一个覆盖组定义文件 `covergroup.svh`，定义功能覆盖率（functional coverage）的采样点——包括基本覆盖点（coverpoint）和交叉覆盖点（cross）。覆盖组使用 SystemVerilog 语法编写，评测阶段通过商用仿真器（VCS）编译和采样，开发/验证阶段通过组委会提供的**覆盖率模拟工具**（参见附件 C）在 Verilator 侧输出相同格式的覆盖率反馈。

开发和评测使用**相同的覆盖点定义**，仅采集实现方式不同。

A.2 通用规范

1. **采样时机**：所有覆盖点统一使用 `@(posedge clk)` 采样
2. **覆盖点类型**：每个 DUT 的覆盖组包含以下五类覆盖点：
 - 基本功能覆盖点
 - 数据边界覆盖点
 - 条件组合覆盖点
 - 时序窗口覆盖点（`sequential`，用于“需持续保持/完整时序才能达成”的功能窗口）
 - 多变量交叉覆盖点
3. **覆盖点总数**：26~120 个（含交叉覆盖点）

4. 文件名: covergroup.svh

A.3 覆盖点类型定义

A.3.1 基本功能覆盖点

覆盖 DUT 的基本操作模式、状态值或功能编码。

```
opcode: coverpoint instr[6:0];  
state: coverpoint fsm_state;
```

A.3.2 数据边界覆盖点

覆盖数据通路的边界值、特殊值。

```
data_val: coverpoint data_out {  
    bins zero    = {0};  
    bins max     = {32'hFFFF_FFFF};  
    bins small   = {[1:15]};  
}
```

A.3.3 条件组合覆盖点

覆盖控制路径的分支条件取值组合。

```
branch_taken: coverpoint branch_taken_signal;
```

A.3.4 时序窗口覆盖点 (sequential)

覆盖“需持续保持、完整时序序列达成”的功能窗口，绑定时序达成/结果正确性等派生信号，命中的前提是 Agent 构造出持续正确的激励序列。该类覆盖点通常配合 coverage_meta.json 里 type: "sequential"。评测端统一按“仅上报 bin 是否命中”掩码，不向 Agent 泄漏触发该 bin 的中间派生信号原始值。

```
win_mode: coverpoint win_mode {          // 完整窗口按 mode 达成  
    bins mode0 = {2'd0};  
    bins mode1 = {2'd1};  
}  
win_len: coverpoint win_len {           // 窗口长度档（具体值保密）  
    bins len0 = {2'd0};  
    bins len1 = {2'd1};  
}
```

注：win_mode / win_len 等派生信号属于**评测机制内部信号**，只在仿真/覆盖采集侧存在，不应写入 dut_spec.md。

A.3.5 交叉覆盖点

覆盖多个信号的组合。

```
op_f3: cross opcode, funct3;  
bt_cond: cross branch_taken, branch_condition;
```

A.4 覆盖点难度分层

难度	类型	说明	随机基线可达率（参考）
易	基本功能覆盖点	直接覆盖状态/操作码等	>70%
中	数据边界覆盖点	需要特定数据值才能覆盖	30%~70%
难	条件组合覆盖点	需要组合条件才能覆盖	<30%
最难	多变量交叉覆盖点	需要多个条件同时满足	<10%

A.5 完整 covergroup.svh 示例

以下是一个小型 ALU（算术逻辑单元）DUT 的完整覆盖组定义示例，覆盖 basic / boundary / condition / cross 四类（sequential 时序窗口见 A.3.4，此处从略），覆盖点总数 26 个（含 3 个交叉覆盖点）：

```
// covergroup.svh - 示例ALU覆盖率定义
// 许可: MIT License
```

```
covergroup alu_cg @(posedge clk);

    // ===== 基本功能覆盖点 (8 bins) =====
    opcode: coverpoint alu_op {
        bins add    = {4'b0000};
        bins sub    = {4'b0001};
        bins and_   = {4'b0010};
        bins or_    = {4'b0011};
        bins xor_   = {4'b0100};
        bins sll    = {4'b0101};
        bins srl    = {4'b0110};
        bins sra    = {4'b0111};
    }

    // ===== 数据边界覆盖点 (4 bins) =====
    result_zero: coverpoint (alu_result == 32'h0) {
        bins false = {1'b0};
        bins true  = {1'b1};
    }

    result_neg: coverpoint alu_result[31] {
        bins pos = {1'b0};
        bins neg = {1'b1};
    }

    // ===== 基本功能覆盖点 (4 bins) =====
    carry_out: coverpoint carry_flag {
        bins zero = {1'b0};
        bins one  = {1'b1};
    }

    overflow: coverpoint overflow_flag {
        bins no  = {1'b0};
        bins yes = {1'b1};
    }

    // ===== 条件组合覆盖点 (4 bins) =====
    carry_overflow: coverpoint {carry_flag, overflow_flag} {
        bins no_carry_no_overflow = {2'b00};
        bins no_carry_overflow    = {2'b01};
        bins carry_no_overflow    = {2'b10};
        bins carry_overflow       = {2'b11};
    }

    // ===== 交叉覆盖点 (6 bins) =====
    op_zero: cross opcode, result_zero {
        bins add_nz = binsof(opcode.add) && binsof(result_zero.false);
        bins add_z  = binsof(opcode.add) && binsof(result_zero.true);
        bins sub_nz = binsof(opcode.sub) && binsof(result_zero.false);
    }

    op_carry: cross opcode, carry_out {
        bins add_carry = binsof(opcode.add) && binsof(carry_out.one);
    }
}
```

```

        bins add_nocarry = binsof(opcode.add) && binsof(carry_out.zero);
    }

    zero_neg_over: cross result_zero, result_neg, overflow {
        bins z_n_o = binsof(result_zero.false) && binsof(result_neg.neg) && binsof(overflow.yes);
    }
endgroup

// 注意：实际使用时需在 DUT 顶层模块中实例化该 covergroup:
//  alu_cg cg_inst = new();
//  cg_inst.sample();
// 采样触发由 @(posedge clk) 自动控制

```

覆盖点清单：

覆盖点名	类型	难度	bin数
opcode	基本功能覆盖点	易	8
result_zero	数据边界覆盖点	中	2
result_neg	数据边界覆盖点	中	2
carry_out	基本功能覆盖点	易	2
overflow	基本功能覆盖点	易	2
carry_overflow	条件组合覆盖点	难	4
op_zero (cross)	交叉覆盖点	难	3
op_carry (cross)	交叉覆盖点	难	2
zero_neg_over (cross)	交叉覆盖点	最难	1

A.6 DUT 功能规格说明（DUT Spec）

每个 DUT 附带一份功能规格说明文档（dut_spec.md），包含以下内容：

DUT 名称

功能概述

[一段话描述DUT的整体功能]

接口信号

信号名	方向	位宽	说明
---	---	---	---
clk	in	1	时钟
rst_n	in	1	复位，低有效
...	...	...	...

操作模式

- 模式A: [描述]
- 模式B: [描述]

激励接口说明

[描述如何向该DUT发送激励，激励的格式和含义]

DUT spec 在推理阶段通过 `InferenceInterface.__init__()` 的 `dut_spec_path` 参数传递给参赛者算法，供其理解 DUT 功能以辅助决策（例如供 LLM 读取）。

A.7 覆盖率反馈接口

评测环境每步仿真后，将覆盖率信息通过 `InferenceInterface.predict()` 的以下参数传递给参赛者算法：

- `coverage_state`: `np.ndarray` — 各覆盖点的已覆盖状态（二值向量，`shape=(total_bins,)`, `dtype=float32`, 1.0 表示已覆盖）
- `step`: `int` — 当前仿真步数（0-indexed）
- `max_steps`: `int` — 最大仿真步数

附加信息（可通过 `coverage_meta.json` 辅助提取）：

- 覆盖点总数： `len(coverage_state)`
- 已覆盖点数： `np.sum(coverage_state)`
- 当前覆盖率： `np.sum(coverage_state) / len(coverage_state)`
- 覆盖点类型区分：参考 `coverage_meta.json` 中的 `type` 字段
- **bin 顺序规则**： `coverage_state` 的 bin 按 `coverage_meta.json` 中 `coverpoints` 数组顺序展开，各覆盖点的 `bins[]` 依次拼接。例如第 0~7 位对应 `opcode` 的 8 个 bin，第 8~9 位对应 `result_zero`，以此类推

A.8 使用约束

- 参赛者**不得修改** `covergroup.svh` 文件
- 开发/验证阶段通过覆盖率模拟工具（附件 C）获取与评测阶段一致的覆盖率反馈
- 所有开源 DUT 使用 BSD / MIT / Apache 许可，详见各组件源码中的 `LICENSE` 文件

第三部分 附件 B：Docker 提交规范与推理接口标准

B.1 概述

组委会提供**Docker 基础镜像**（含 InferenceInterface 骨架代码、覆盖率模拟工具），参赛队在此基础镜像之上开发自己的算法并提交最终的 Docker 镜像（不暴露源码）。评测脚本将导入镜像中的 `inference` 模块，调用其 `predict()` 方法驱动激励生成。

B.2 镜像结构要求

```
/
├── app/
│   └── inference/
│       ├── __init__.py          # 推理入口（由参赛者实现）
│       ├── model/              # 模型权重文件
│       └── requirements.txt     # 额外 Python 依赖（可选）
└── entrypoint.sh               # 容器入口脚本
```

B.3 推理接口标准

参赛者必须在 `/app/inference/__init__.py` 中实现以下接口：

```
import numpy as np

class InferenceInterface:
    """推理接口——由评测脚本调用"""

    def __init__(self, dut_spec_path: str, covergroup_path: str):
        """
        初始化推理环境。

        Args:
            dut_spec_path: DUT 功能规格说明文件路径 (Markdown)
            covergroup_path: 覆盖组定义文件路径 (SystemVerilog)
        """
        pass

    def predict(
        self,
        coverage_state: np.ndarray,
        step: int,
        max_steps: int,
    ) -> np.ndarray:
        """
        根据当前覆盖率状态，返回下一个激励动作。

        Args:
            coverage_state: 当前各覆盖点已覆盖状态 (shape=(N,)), dtype=float32
                           1.0 表示已覆盖, 0.0 表示未覆盖
            step: 当前步数 (0-indexed)
            max_steps: 最大仿真步数

        Returns:
            action: 下一个激励动作，一维 numpy 向量 (shape=(D,)), dtype=float32
                   D 为动作空间维度，由 DUT spec 定义
                   评测脚本中的仿真适配层负责将 action 解码为 DUT 引脚激励
        """
        raise NotImplementedError
```

约束：

- predict() 单次调用耗时建议 $\leq 30\text{ms}$ （评测环境 CPU 上）。该值与 2 小时墙钟上限、3 个评测 DUT、每 DUT 5 万 cycles 的约束对应： $2\text{h} \div 3 \div 5\text{万} \approx 44\text{ms/cycle}$ ，扣除 VCS 仿真开销后留给 predict 的预算约 30ms。超时不影响评分但会减少可完成的 cycles 数，从而影响最终覆盖率
- 评测脚本在单线程中循环调用 predict()，不并发
- __init__() 允许较长的初始化时间（如加载模型权重），不计入评测时间
- **严格禁止**在 predict() 中修改模型权重参数；允许在推理时根据覆盖率反馈动态调整prompt策略或Agent规划（prompt engineering层面的适应性调整）。不得更新模型权重

激励解码约定

组委会评测脚本负责将 predict() 返回的 action 向量解码为 DUT 引脚激励（参赛者无需实现解码逻辑）：

1. predict() 返回一个 numpy 向量（np.ndarray，shape=(A,), dtype=float32），A 为动作空间维度
2. 每个 DUT 的 dut_spec.md 定义了从 action 向量到 DUT 引脚信号的映射规则
3. 评测脚本根据 dut_spec.md 的映射表，将 action 向量转换为仿真器输入信号
4. 解码规则随基础镜像一同提供，包含在评测脚本的仿真适配层中

参赛者约束：

- action 向量的取值和语义完全由参赛者自定——评测脚本只按 dut_spec.md 的映射规则解码
- 参赛者应确保 predict() 输出的 action 在 DUT 的有效激励范围内
- 动作空间维度 D 由 DUT spec 中的"激励接口说明"定义，可理解为动作向量中独立控制通道的数量。对于多参数激励（如操作码 + 多个操作数），所有控制通道展开为一维向量，解码规则和取值范围见 DUT spec

B.4 评测脚本调用流程

```
import sys
sys.path.insert(0, '/app')

from inference import InferenceInterface

# 步骤1：初始化
inference = InferenceInterface(
    dut_spec_path='/dut/spec.md',
    covergroup_path='/dut/covergroup.svh',
)

# 步骤2：仿真循环
coverage_state = np.zeros(total_bins, dtype=np.float32)
for step in range(max_steps):
    action = inference.predict(coverage_state, step, max_steps)
    coverage_state, coverage, done = simulator.step(action)
    if done:
        break
```

B.5 镜像构建与提交

参赛队基于组委会提供的基础镜像构建（FROM <组委会基础镜像>）：

```
FROM <组委会基础镜像>

# 安装额外依赖（如有）
COPY app/inference/requirements.txt /tmp/extra_requirements.txt
RUN pip install --no-cache-dir -r /tmp/extra_requirements.txt || true

# 复制推理代码和模型
COPY app/ /app/
```

提交物：

- Docker 镜像 tar 包：docker save -o submission.tar <image_name>:<tag>
- 镜像内 /app/inference/__init__.py 必须实现标准 InferenceInterface

B.6 调用示例

参赛者在开发阶段可自行编写调用代码验证接口，例如：

```
import sys
sys.path.insert(0, '/app')
from inference import InferenceInterface
import numpy as np

inf = InferenceInterface('dut_spec.md', 'covergroup.svh')
state = np.zeros(26, dtype=np.float32) # 与 example ALU 的 26 bins 对应
action = inf.predict(state, step=0, max_steps=50000)
action = inf.predict(state, step=0, max_steps=50000)
print(f"Action: {action}")
```

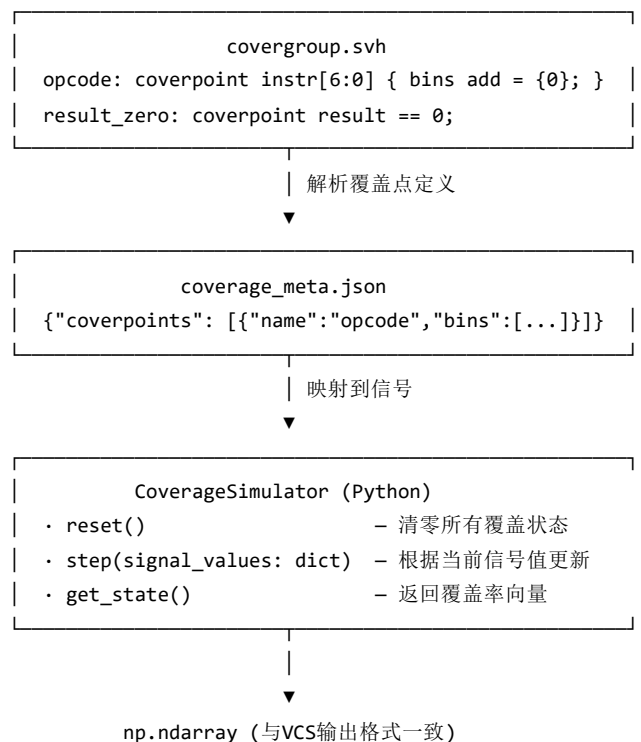
第四部分 附件 C：覆盖率模拟工具规范

C.1 概述

组委会提供**覆盖率模拟工具**（Coverage Simulator），一个 Python 工具包，使参赛者在开发/验证阶段（Verilator 环境）获得与评测阶段（VCS 环境）一致的覆盖率反馈。

该工具读取 covergroup 定义，在 Verilator 仿真过程中观测 DUT 信号，按 covergroup 规则计算各覆盖点的命中状态，输出与评测阶段相同格式的 numpy 向量。

C.2 工作原理



C.3 API 定义

```
import numpy as np
from typing import Dict

class CoverageSimulator:
    """在 Verilator 端模拟 covergroup 覆盖率采集。

    接口与评测环境一致，开发/验证时替换 VCS 的实际 covergroup 采样。
    """

    def __init__(self, meta_path: str):
        """
        初始化覆盖率模拟器。

        Args:
            meta_path: coverage_meta.json 文件路径
        """
        pass

    def reset(self) -> np.ndarray:
        """
        复位所有覆盖点的命中状态。

        Returns:
            coverage_state: shape=(total_bins,), dtype=float32, 全零向量
        """
        pass

    def step(self, signal_values: Dict[str, int]) -> np.ndarray:
        """
        根据当前周期的DUT信号值，更新覆盖点命中状态。

        Args:
            signal_values: 信号名到当前周期值的映射
                {'instr': 0x00100093, 'alu_result': 0, 'carry': 0, ...}

        Returns:
            coverage_state: 更新后的覆盖率向量
                shape=(total_bins,), dtype=float32
                1.0 表示该覆盖点已命中过，0.0 表示从未命中
        """
        pass

    @property
    def coverage(self) -> float:
        """当前总覆盖率 (0.0 ~ 1.0)。"""
        pass

    @property
    def covered_bins(self) -> int:
        """已命中的覆盖点数量。"""
        pass

    @property
    def total_bins(self) -> int:
```

```
"""覆盖点总数。"""  
pass
```

C.4 在仿真循环中使用

参赛者在开发脚本中，每个仿真周期调用一次 `step()`：

```
from coverage_simulator import CoverageSimulator  
import verilator_binding # Verilator C++ 到 Python 的绑定  
  
# 初始化  
cov_sim = CoverageSimulator("dut/coverage_meta.json")  
state = cov_sim.reset()  
  
# 仿真循环  
for cycle in range(max_cycles):  
    # 1. 生成激励（参赛者算法）  
    action = agent.predict(state, cycle, max_cycles)  
  
    # 2. 应用到 DUT  
    verilator_binding.apply_stimulus(action)  
  
    # 3. 读取 DUT 信号值  
    signals = verilator_binding.read_signals(['instr', 'alu_result', 'carry', ...])  
  
    # 4. 更新覆盖率（用模拟工具替代 VCS covergroup）  
    state = cov_sim.step(signals)  
  
    # 5. 计算奖励  
    reward = cov_sim.coverage - prev_coverage  
    prev_coverage = cov_sim.coverage
```

C.5 coverage_meta.json 格式

以下为格式示例（片段，非完整定义；完整定义见 `example/dut/coverage_meta.json`）：

```
{
  "dut_name": "example_alu",
  "total_bins": 26,
  "coverpoints": [
    {
      "name": "opcode",
      "type": "basic",
      "signal": "alu_op",
      "bins": [
        {"name": "add", "match": 0},
        {"name": "sub", "match": 1}
      ]
    },
    {
      "name": "result_zero",
      "type": "boundary",
      "signal": "alu_result",
      "bins": [
        {"name": "nonzero", "match": "x != 0"},
        {"name": "zero", "match": "x == 0"}
      ]
    },
    {
      "name": "op_carry",
      "type": "cross",
      "parents": ["opcode", "carry_out"],
      "cross_bins": [
        {"combinations": [[0,1]], "desc": "add_carry"},
        {"combinations": [[0,0]], "desc": "add_nocarry"}
      ]
    }
  ]
}
```

C.6 覆盖点格式说明

non-cross 覆盖点 (basic / boundary / condition / sequential):

- **signal**: DUT 中可观测的信号名, [M:N] 表示位选 (如 `alu_result[31]` 表示符号位)。对于 `condition` 类型, **signal** 也可以是逻辑组合后的**派生信号名** (如 `carry_overflow` 表示 `{carry_flag, overflow_flag}` 的组合值), 派生信号不在 DUT 顶层直接存在, 参赛者需在仿真循环中显式计算并注入到 `signal_values` 字典中。对于 `sequential` 类型, 其 **signal** 为窗口达成/结果正确性等派生信号 (如 `win_mode / win_len / win_ctl`), 由仿真模型派生, 采集侧同样只上报“bin 是否命中”。
- **bins[].match**: 两种模式——**整数**表示信号精确相等匹配; **字符串** `"x == 0"` / `"x != 0"` 表示边界条件匹配 (CoverageSimulator 内置支持这两种表达式, 不解析通用布尔表达式)
- **difficulty** (可选): 覆盖点难度标签 (easy/medium/hard/hardest), 由 CoverageSimulator 忽略, 供参赛者参考
- **num_bins** (可选): 覆盖点 bin 数量, 与 `bins[]` 数组长度一致

cross 覆盖点:

- **parents**: 引用父覆盖点名称列表 (必须与 `coverpoints` 中已定义的覆盖点名称一致)
- **cross_bins[].combinations**: 父覆盖点 bin 索引的组合列表, 每个组合为 `[bin_idx_p0, bin_idx_p1, ...]`, 对应 `parents` 数组的顺序
- 当前周期内若某个 combination 中所有父 bin 均已命中, 则该 cross bin 标记为已覆盖

C.7 工具交付物

组委会以 Python 包形式提供覆盖率模拟工具：

```
coverage_simulator/
├── __init__.py          # 导出 CoverageSimulator
├── core.py              # 覆盖率计算核心逻辑
└── README.md           # 使用说明
```

参赛者可通过 pip 安装或直接嵌入开发脚本。

C.8 与评测环境的一致性保证

方面	开发/验证（模拟工具）	评测（VCS）	一致性
覆盖点定义	coverage_meta.json	covergroup.svh	同源转换
bin 定义	JSON 表达式	SystemVerilog bins	等价
采样时机	每步骤调用一次	@(posedge clk)	每 cycle 等价
输出格式	np.ndarray (float32)	np.ndarray (float32)	完全相同
cross 匹配时机	当前周期	当前周期	✅ 等价——均以同一 cycle 的信号值判断
高级特性兼容性	不支持 iff 条件采样、option.per_instance 等进阶特性	原生支持	✅ 评测 DUT 的 covergroup.svh 不使用这些特性，确保一致性

组委会在赛前完成至少一个完整 DUT 的双工具交叉验证（模拟工具输出 vs VCS 输出逐 cycle 对比），确保覆盖率趋势一致。

***本赛题指南未尽问题，见赛题 O&A 文件**